[MP2] checker 評分標準與 spec 不一致

2 2

楊竣凱 (Yang, Chun kai)

All Sections

為了實現 alloc & free 真 O(1) 而非 amortized O(1)，沒辦法一次完成 slab 內 space linked list initialized 的過程，在 print_kmem_cache 中輸出的 as_ptr 將不一定會是下一個 space 的 address，根據 spec 的敘述也不該限定為 address，但 checker 會因此判此為 FAIL。

已經確認只有跟 print_kmem_cache 有關的 testcase 會出錯。

[更1] 在 print_kmem_cache 順便 init linked list 狀態後會變為全 OK，可以確認為 checker 問題

[更2] 發現新的 checker 在 [partial slabs] 中如果包含全空的 slabs 會 FAIL，與 spec 敘述和範例（第四個）皆不符合

以下附上一般結果（一）與不先 init （二）mp2 的截圖

（一）

```
xv6 kernel is booting

[MP2] Slab size: 3
[FILE] fileinit
[SLAB] New kmem_cache (name: file, object size: 504 bytes, at: 0x0000000087f59000, max objects per slab: 8, support in cache obj: 7) is created
hart 1 starting
hart 2 starting
[FILE] filealloc
[SLAB] Alloc request on cache file
[SLAB] Object 0x0000000087f59060 in slab 0x0000000087f59000 (file) is allocated and initialized
init: starting sh
[FILE] filealloc
[SLAB] Alloc request on cache file
[SLAB] Object 0x0000000087f59258 in slab 0x0000000087f59000 (file) is allocated and initialized
[FILE] fileclose
[SLAB] Free 0x0000000087f59258 in slab 0x0000000087f59000 (file)
[SLAB] End of free
$ mp2
[SLAB] kmem_cache { name: file, object_size: 504, at: 0x0000000087f59000, in_cache_obj: 7 }
[SLAB]   [ cache slabs ]
[SLAB]     [ slab 0x0000000087f59000 ] { freelist: 0x0000000087f59258, nxt: 0x0000000000000000 }
[SLAB]       [ idx 0 ] { addr: 0x0000000087f59060, as_ptr: 0x0000000900000003, as_obj: {tp: 3, ref: 9, readable: 1, writable: 1, pipe: 0x0505050505050505, ip: 0x0000
[SLAB]       [ idx 1 ] { addr: 0x0000000087f59258, as_ptr: 0x0000000087f59450, as_obj: {tp: -2013948848, ref: 0, readable: 6, writable: 0, pipe: 0x0505050505050505,
[SLAB]       [ idx 2 ] { addr: 0x0000000087f59450, as_ptr: 0x0000000087f59648, as_obj: {tp: -2013948344, ref: 0, readable: 5, writable: 0, pipe: 0x0505050505050505,
[SLAB]       [ idx 3 ] { addr: 0x0000000087f59648, as_ptr: 0x0000000087f59840, as_obj: {tp: -2013947840, ref: 0, readable: 4, writable: 0, pipe: 0x0505050505050505,
[SLAB]       [ idx 4 ] { addr: 0x0000000087f59840, as_ptr: 0x0000000087f59a38, as_obj: {tp: -2013947336, ref: 0, readable: 3, writable: 0, pipe: 0x0505050505050505,
[SLAB]       [ idx 5 ] { addr: 0x0000000087f59a38, as_ptr: 0x0000000087f59c30, as_obj: {tp: -2013946832, ref: 0, readable: 2, writable: 0, pipe: 0x0505050505050505,
[SLAB]       [ idx 6 ] { addr: 0x0000000087f59c30, as_ptr: 0x0000000000000000, as_obj: {tp: 0, ref: 0, readable: 1, writable: 0, pipe: 0x0505050505050505, ip: 0x0505
[SLAB] print_kmem_cache end
$ qemu-system-riscv64: terminating on signal 15 from pid 591 (make)
```

（二）

```
*** Now run 'gdb' in another window.

xv6 kernel is booting

[MP2] Slab size: 3
[FILE] fileinit
[SLAB] New kmem_cache (name: file, object size: 504 bytes, at: 0x0000000087f59000, max objects per slab: 8, support in cache obj: 7) is create
hart 2 starting
hart 1 starting
[FILE] filealloc
[SLAB] Alloc request on cache file
[SLAB] Object 0x0000000087f59060 in slab 0x0000000087f59000 (file) is allocated and initialized
init: starting sh
[FILE] filealloc
[SLAB] Alloc request on cache file
[SLAB] Object 0x0000000087f59258 in slab 0x0000000087f59000 (file) is allocated and initialized
[FILE] fileclose
[SLAB] Free 0x0000000087f59258 in slab 0x0000000087f59000 (file)
[SLAB] End of free
$ mp2
[SLAB] kmem_cache { name: file, object_size: 504, at: 0x0000000087f59000, in_cache_obj: 7 }
[SLAB]    [ cache slabs ]
[SLAB]      [ slab 0x0000000087f59000 ] { freelist: 0x0000000087f59258, nxt: 0x0000000000000000 }
[SLAB]        [ idx 0 ] { addr: 0x0000000087f59060, as_ptr: 0x0000000900000003, as_obj: {tp: 3, ref: 9, readable: 1, writable: 1, pipe: 0x05050
[SLAB]        [ idx 1 ] { addr: 0x0000000087f59258, as_ptr: 0x0000000087f59450, as_obj: {tp: -2013948848, ref: 0, readable: 6, writable: 0, pip
[SLAB]        [ idx 2 ] { addr: 0x0000000087f59450, as_ptr: 0x0000000000000000, as_obj: {tp: 0, ref: 0, readable: 5, writable: 0, pipe: 0x05050
[SLAB]        [ idx 3 ] { addr: 0x0000000087f59648, as_ptr: 0x0505050505050505, as_obj: {tp: 84215045, ref: 84215045, readable: 5, writable: 5,
[SLAB]        [ idx 4 ] { addr: 0x0000000087f59840, as_ptr: 0x0505050505050505, as_obj: {tp: 84215045, ref: 84215045, readable: 5, writable: 5,
[SLAB]        [ idx 5 ] { addr: 0x0000000087f59a38, as_ptr: 0x0505050505050505, as_obj: {tp: 84215045, ref: 84215045, readable: 5, writable: 5,
[SLAB]        [ idx 6 ] { addr: 0x0000000087f59c30, as_ptr: 0x0505050505050505, as_obj: {tp: 84215045, ref: 84215045, readable: 5, writable: 5,
[SLAB] print_kmem_cache end
$ qemu-system-riscv64: terminating on signal 15 from pid 238 (make)
```

Search entries or author | Unread | Subscribe

Reply



陳楚融 (CHEN, CHU-RONG) (https://cool.ntu.edu.tw/courses/46296/users/105887)

Tuesday

+1 也遇到同樣問題
更新評分標準之前是不是應該更仔細檢查測試系統或是說明判定標準

Edited by 陳楚融 (CHEN, CHU-RONG) (https://cool.ntu.edu.tw/courses/46296/users/105887) on Apr 1 at 8:57pm

Reply



楊子慶 (YANG, TZU-CHING) (https://cool.ntu.edu.tw/courses/46296/users/229158)

Tuesday

同學們好：

很抱歉由於未說明清楚導致困擾。

由於直譯器會在 print_kmem_cache 時檢查 slab 內的物件是否以鏈結串列的方式串連，無論是否實現 O(1)，請在 print_kmem_cache 建立完整的鏈結序列。這部分並不會納入 O(1) 的評分考量。

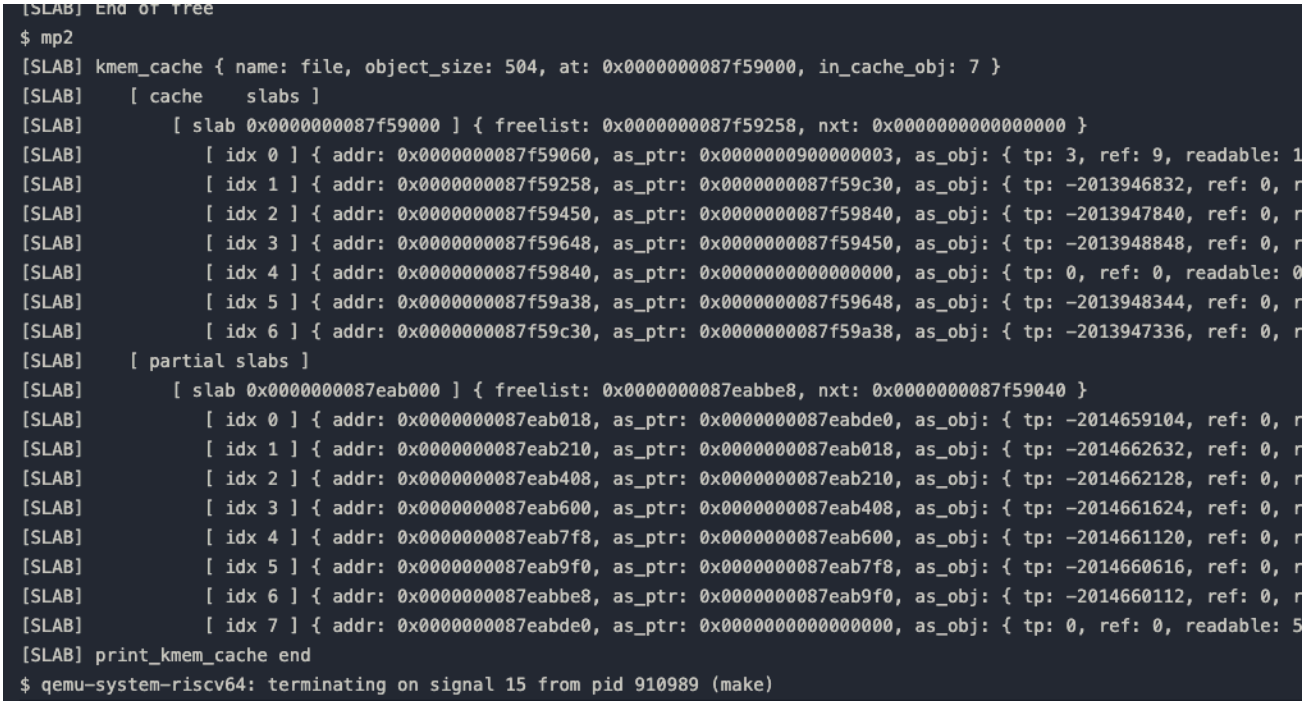
另 partial slab 裡面所有物件都處於可配置的情況下並不會被判定為 FAIL，舉以下範例：

// mytest.txt

echo Ok | gah . 10 | oak end
mp2

運行測試：

./mp2.sh test custom
輸出：



終端機輸出（由於 custom 不計分，故只寫 0% + Score: 0/0 不報錯）：

./mp2.sh testcase custom
== Test custom/mytest (0%) ==
(3.5s)

Score: 0/0

若有更多問題請不吝提出，感謝。

子慶

↰ **Reply**